

The Friday Handoff

A Container Triage at Cordwell Home & Hardware

Week 7, Module 01 discussion exercise. Teams of 3. Target 35 to 45 minutes.

The Situation

It is Monday morning at Cordwell Home & Hardware. Riley, a junior engineer on your platform team, spent last week doing the company's very first containerization work on the review-summarizer service, then left Friday at 5 PM for a two-week vacation – reachable only by patchy resort Wi-Fi.

Your team of three has inherited the handoff. In three rounds you will judge which of last quarter's incidents containers would have prevented, review the Dockerfile Riley left behind, and clear the Monday morning ticket queue. Everything you need is on the slides you just saw. All data and services are synthetic.

How This Works

- Discuss in teams of 3.
- You may reference the slides at any time. That is encouraged, not cheating.
- Each round ends with a written deliverable. Be ready to defend one decision per round in the debrief.

Round 1: Would a Container Have Saved Us? (9 minutes)

Below are four real-feeling incidents from Cordwell's last quarter. For each one, decide as a team: would containerizing the service have prevented this incident, yes or no? Write a one sentence justification. Warning: at least one of these is a trap, because containers are not magic.

#	Incident	Yes or no	Why (one sentence)
1	Tests pass on an engineer's laptop with numpy 2.4.1 but fail in CI, which resolved numpy 2.3.5. Two hours lost to 'works on my machine.'		
2	review-summarizer runs fine on a Mac, but on the Linux staging host it crashes on startup: a native library one of the wheels depends on cannot be found on that OS.		
3	An engineer loads the larger summarizer model variant and the process is killed: the laptop simply does not have enough memory for it.		
4	Two engineers get different summaries for the same review batch. It turns out one of them had a stale copy of the seeded reviews CSV in their local checkout.		

Deliverable: yes or no plus a one sentence justification for each incident. Bonus point from the Skeptic: for every 'yes,' name the cheaper tool that solves part of the problem, and say what the container adds on top of it.

Round 2: The Dockerfile Riley Left Behind (13 minutes)

Riley's handoff note reads: 'Dockerfile works! Builds are kind of slow (about 9 minutes even for a one line change) but I think that is just Docker.' Here is the repo layout and the Dockerfile.

Repo layout

```
review-summarizer/  
  app/                # 12 Python files, the actual service  
  requirements.txt     # pinned dependencies  
  data/reviews_full.csv # 2.1 GB synthetic review dump  
  models/summarizer.ckpt # 850 MB experiment checkpoint  
  notebooks/          # exploration notebooks  
  .git/               # about 400 MB of history  
  Dockerfile           # below. There is no .dockerignore.
```

Riley's Dockerfile

```
FROM python:latest  
ADD . /app  
WORKDIR /app  
RUN pip install -r requirements.txt  
RUN pip cache purge  
CMD python -m app.serve
```

Your tasks

- Task A: Find every issue you can. There are at least five distinct ones. For each, say what it costs Riley: build time, image size, security, or correctness.
- Task B: Agree on your top three fixes, ranked by impact, and write the corrected Dockerfile lines for them. Slides 8 to 10 have everything you need.
- Task C, the prediction: Riley edits one line of `app/serve.py` and rebuilds. Walk through the Dockerfile line by line and predict exactly which layers rebuild and why the build takes 9 minutes. Then predict the rebuild time after your fixes.

Deliverable: the ranked top three fixes with corrected lines, plus your layer by layer answer to Task C.

Round 3: The Monday Morning Ticket Queue (11 minutes)

Three tickets are waiting. For each, write the root cause in one sentence and the exact fix: the corrected command or flag, not just a description. The slides on run options, volumes, networking, and cross-platform builds are your friends.

Ticket 101: 'The API is a ghost'

```
$ docker run -d --name rs review-summarizer:v1  
$ docker ps  
CONTAINER ID   IMAGE                  STATUS              NAMES  
9f3c1a2b77e0   review-summarizer:v1 Up 40 seconds      rs  
$ curl http://localhost:8000/health  
curl: (7) Failed to connect to localhost port 8000: Connection refused
```

The service logs show it listening happily on port 8000 inside the container. Root cause? Exact fixed command?

Ticket 102: 'The overnight batch that never was'

Riley ran the batch summarizer overnight to process the seeded reviews file:

```
$ docker run --rm review-summarizer:v1 \
  python -m app.batch --in /data/reviews.csv --out /out
```

Two problems this morning. First, the batch reported success, but the summaries in /out are nowhere to be found on the host. Second, a bug in the batch code overwrote the input CSV inside the container, and Riley wants a guarantee that can never happen to the source data again. Root causes? One corrected command that fixes both?

Ticket 103: 'It will not even start'

Riley built the image on a MacBook, pushed it to the registry, and pulled it on Cordwell's amd64 cloud VM:

```
$ docker run registry.example.com/cordwell/review-summarizer:v1
exec /usr/local/bin/python: exec format error
```

The exact same image ran perfectly on the Mac ten minutes earlier. Root cause? Two different fixes from slide 12, and when would you pick each?

Deliverable: for each ticket, a one sentence root cause plus the exact corrected command.

Stretch Goal (if you finish early)

The elevator brief: Cordwell's director of engineering asks, 'Why are we doing containers instead of just giving every service its own VM?' Draft the two sentence answer your team would actually say out loud, then name one situation where a VM is still the right call. Slide 5 is your source material.

Bring to the Debrief

- Your Round 1 verdict.
- Your number one Dockerfile fix and the before and after rebuild story from Task C.
- The ticket your team solved fastest, and the exact command that closes it.